

Formation : Initiation GIT

**Outil Git Bash & site
gitlab.com**



- **Durée : 1 jours**
 - **Horaires**
 - **Matin : 9h-12H**
 - **Après-midi : 13H30-17H30**
 - **Pose de 15 minutes à 10H45 et 15H45**
 - **Présentation, tour de table**
 - **Votre parcours très rapide**
 - **Vos connaissances sur le sujet**
 - **Vos attentes**
 - **Prérequis**
 - **Bonne connexion internet**
 - **Postes identiques pour les stagiaires**
 - **De préférence double écrans pour le distanciel**
- ## **Début de la formation**
- **Alternance de cours et exercices avec outils**



- **PRESENTATION GIT**
- EXERCICES GIT
- CONCLUSIONS
- LES BRANCHES
- LES CONFLITS DE MERGE
- CONCLUSIONS

Présentation GIT

- Quels sont les objectifs premiers et indispensables d'un outil de gestion de versions ?

Dépôt local GIT

- HISTORISER/VERSIONNER

Historise tout changement de votre code source
Chaque commit fait une photo de votre code à un instant donné

Dépôt distant GIT

- HISTORISER/VERSIONNER

- SAUVEGARDER

Le dépôt distant est indispensable pour sécuriser votre travail (avoir une copie ailleurs que sur un poste local)

- PARTAGER

Le dépôt distant est indispensable pour permettre un travail collaboratif ce qui sera toujours le cas dans un projet de développement car il faut aussi le rendre accessible par les outils d'intégration et de déploiement.

Présentation GIT

- Projet candidat au versionning :

- Est-ce qu'un projet SoapUI Open source est candidat ? (non car tout le projet est dans un seul fichier complexe xml)
- Il est toujours indispensable d'avoir un développement structuré avec plusieurs niveaux d'implémentations de vos fichiers sources afin d'éviter le travail à plusieurs sur le même fichier.

- Architecture du dépôt

Le dossier qui contient un projet GIT contient toujours un répertoire .git qui est la base de données de toutes informations d'historisation de vos fichiers, de vos branches (par défaut : master | main) et de votre url de votre dépôt distant, tout sera disponible en local et répliqué sur un dépôt distant.

Présentation GIT : Documentation

- Documentation de référence

<https://git-scm.com/docs>

- Documentation Book Fr

<https://git-scm.com/book/fr/v2>

- Cheat Sheet

<https://training.github.com/downloads/fr/github-git-cheat-sheet/>

- Visual Cheat Sheet

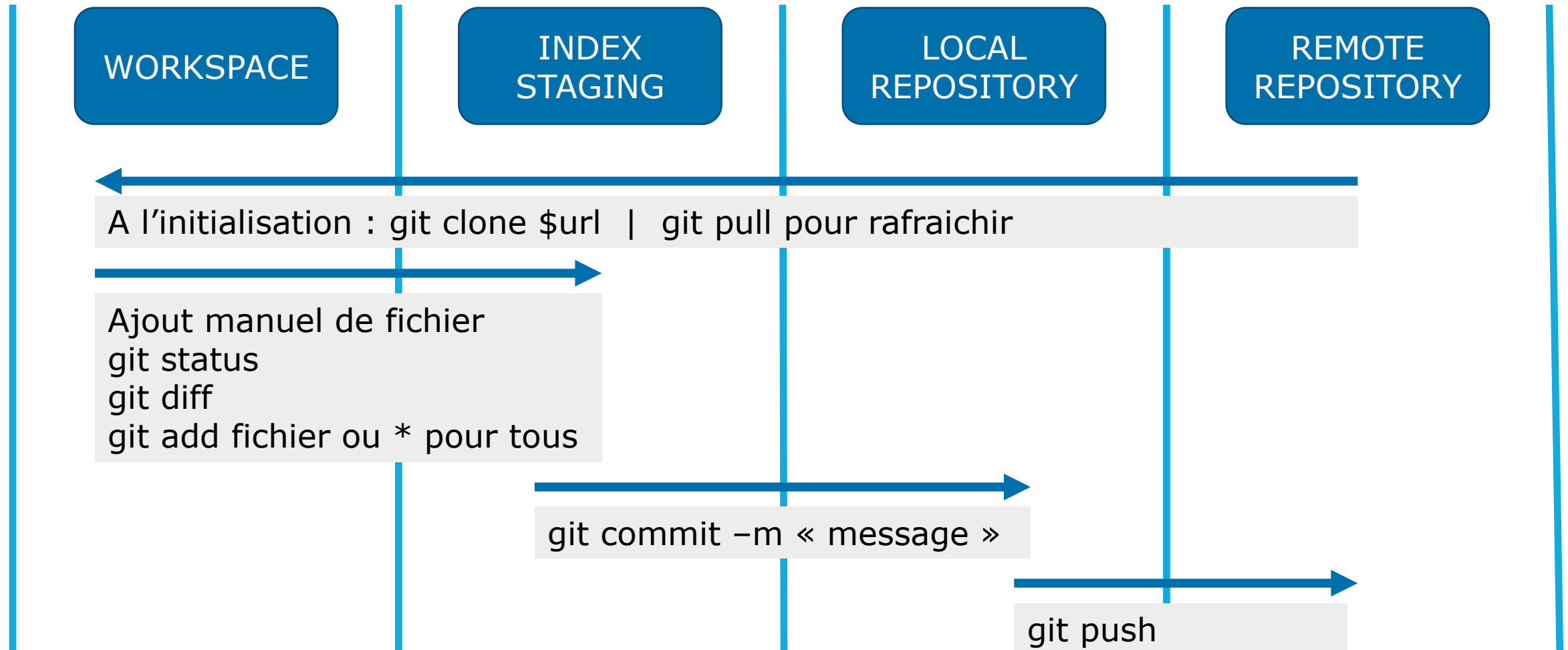
<https://ndpsoftware.com/git-cheatsheet.html>

→ Intéressant pour savoir à quel moment utiliser les diverses commandes GIT

Présentation GIT

- **Un exemple de workflow simple**

En général par simplicité, on crée un repo vide sur le dépôt distant via l'interface Web de GitHub/GitLab, puis on fait un premier clone et on commence à déposer nos fichiers de notre projet. (ADD, COMMIT, PUSH)

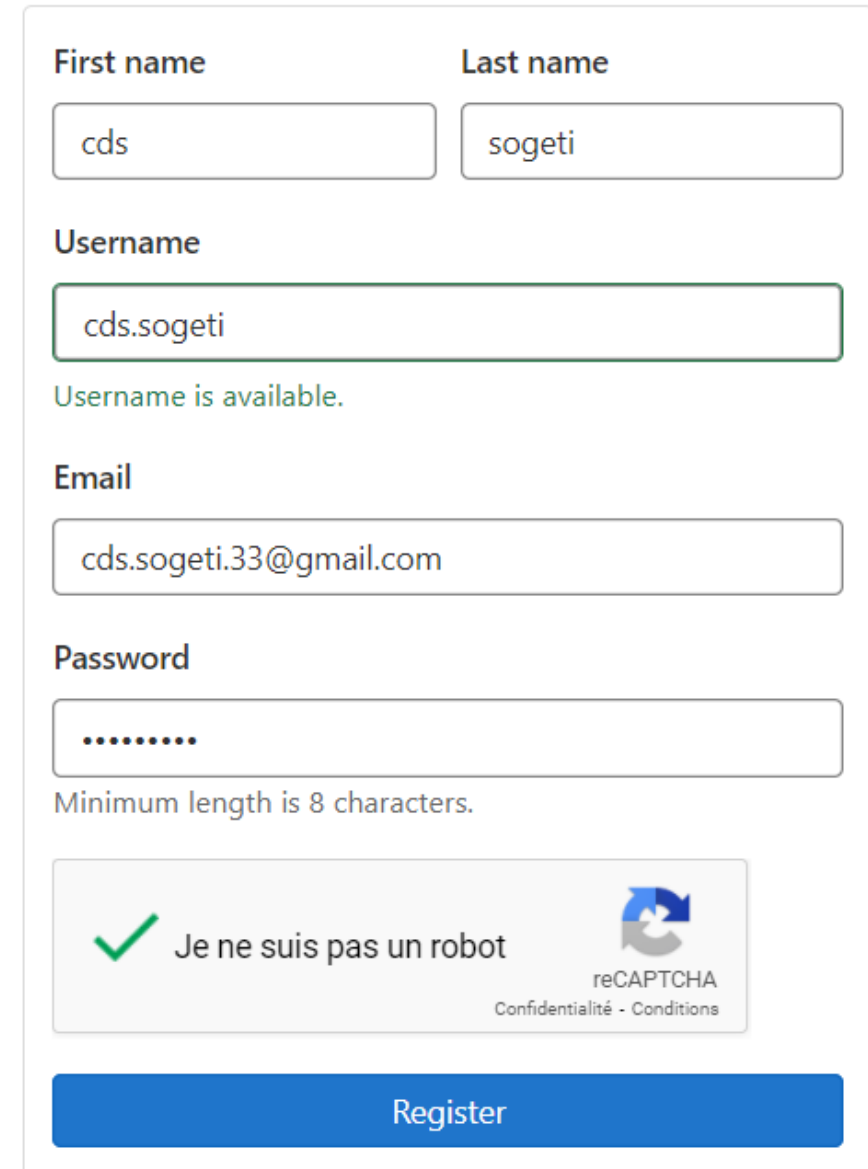
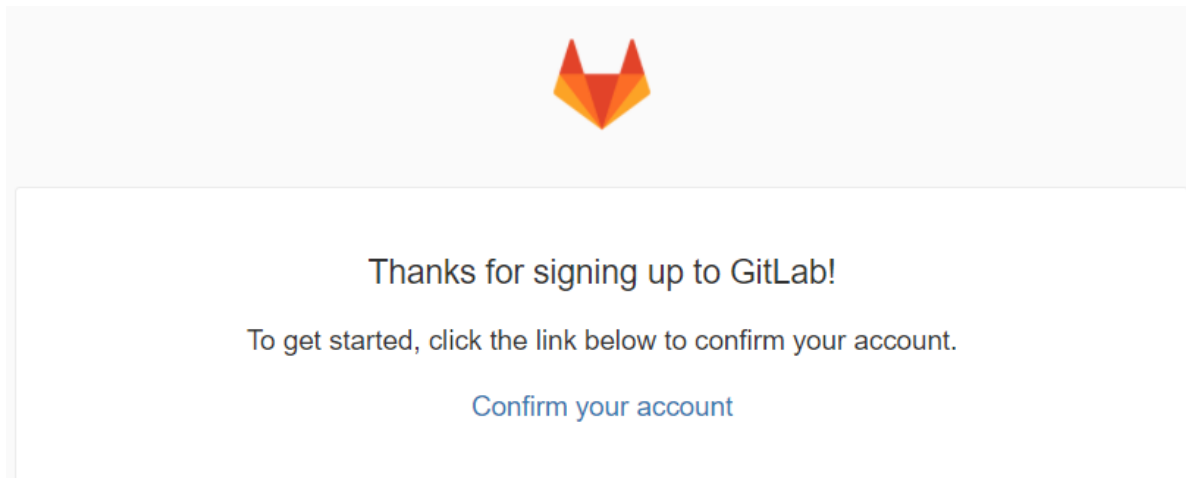


Présentation GIT : Compte GitLab.com

- Création du compte gitlab.com

https://gitlab.com/users/sign_up

- Confirmation du compte depuis votre messagerie

A screenshot of the GitLab sign-up form. It includes fields for "First name" (cds), "Last name" (sogeti), "Username" (cds.sogeti), "Email" (cds.sogeti.33@gmail.com), and "Password" (masked with dots). Below the password field is the text "Minimum length is 8 characters." At the bottom is a blue "Register" button. A reCAPTCHA widget is also present, showing a green checkmark and the text "Je ne suis pas un robot".

Présentation GIT : Compte GitLab.com

1. Your profile

2. Your GitLab group

3. Your first project

Welcome to GitLab,
cds!

To personalize your GitLab experience, we'd like to know a bit more about you.
We won't share this information with anyone.

Role

Software Developer

Who will be using GitLab?

My company or team

Just me

Email updates (optional)

I'd like to receive updates about GitLab via email

I'm signing up for GitLab because:

I want to learn the basics of Git

Continue

1. Your profile

2. Your GitLab group

3. Your first project

Create your group

A group represents your organization in GitLab. Groups allow you to manage users and collaborate across multiple projects.

Group name (your organization)

private-cds-sogeti

You can always edit this later

Group URL

https://gitlab.com/private-cds-sogeti

Visibility level

Private

Create group

Un groupe perso à vous

1. Your profile

2. Your GitLab group

3. Your first project

Create/import your first project

This project will live in your group **private-cds-sogeti**. A project is where you house your files (repository), plan your work (issues), publish your documentation (wiki), and so much more.

Create

Import

Project name

formation

You can always edit this later

Project URL

https://gitlab.com/private-cds-sogeti/formation

Visibility Level ?

Private

Project access must be granted explicitly to each user. If this project is part of a group, access will be granted to members of the group.

Create project

Plutôt nom de projet = formation-git



Présentation GIT : Compte GitLab.com



Get started with GitLab

Ready to get started with GitLab? Follow these steps to set up your workspace, plan and commit changes, and deploy your project.



Set up



Plan & execute



Deploy

Ok, let's go

Vos informations qui seront utiles lors de la configuration de votre client Git Bash

Click sur formation

← → ↺ 🏠 gitlab.com/private-cds-sogeti/formation/-/learn_gitlab



GitLab

Menu

F formation

private-cds-sogeti > formation > Learn GitLab

Exemple de commandes utilisables via git bash

Command line instructions

You can also upload existing files from your computer using the instructions below.

Git global setup

```
git config --global user.name "cds sogeti"
git config --global user.email "cds.sogeti.33@gmail.com"
```

Create a new repository

```
git clone https://gitlab.com/private-cds-sogeti/formation.git
cd formation
git switch -c main
touch README.md
git add README.md
git commit -m "add README"
git push -u origin main
```

Présentation GIT : Client Git Bash

- Téléchargement

<https://git-scm.com/download/win>

En général suivre l'installation avec les options par défaut

Git Bash sera disponible après installation

C'est un terminal en ligne de commande

```
$ git
usage: git [--version] [--help] [-C <path>] [-c <name>=<value>]
        [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
        [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
        [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
        <command> [<args>]

These are common Git commands used in various situations:


start a working area (see also: git help tutorial)
    clone             Clone a repository into a new directory
    init              Create an empty Git repository or reinitialize an existing one


work on the current change (see also: git help everyday)
    add               Add file contents to the index
    mv                Move or rename a file, a directory, or a symlink
    restore            Restore working tree files
    rm                Remove files from the working tree and from the index
    sparse-checkout    Initialize and modify the sparse-checkout


examine the history and state (see also: git help revisions)
    bisect            Use binary search to find the commit that introduced a bug
    diff              Show changes between commits, commit and working tree, etc
    grep              Print lines matching a pattern
    log               Show commit logs
    show              Show various types of objects
    status             Show the working tree status


grow, mark and tweak your common history
    branch             List, create, or delete branches
    commit             Record changes to the repository
    merge              Join two or more development histories together
    rebase             Reapply commits on top of another base tip
    reset              Reset current HEAD to the specified state
    switch             Switch branches
    tag                Create, list, delete or verify a tag object signed with GPG


collaborate (see also: git help workflows)
    fetch              Download objects and refs from another repository
    pull               Fetch from and integrate with another repository or a local branch
    push               Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.
```

Présentation GIT : Client Git Bash

- Configurer une fois vos identifiants depuis les informations de repo sur GitLab
- Lignes de commandes à saisir dans la console Git Bash avec vos propres informations du slide 10

Git global setup

```
git config --global user.name "cds sogeti"  
git config --global user.email "cds.sogeti.33@gmail.com"
```

Présentation GIT : Fichiers indispensables

- **Le README.md**

Ce fichier à la racine du projet dont le contenu est automatiquement affiché sur l'url de votre dépôt distant est indispensable et devrait être obligatoire.

Il contient la description de votre projet et de son utilisation

Il est écrit au format Markdown

(Si besoin, il existe un éditeur ici <https://markdownmonster.west-wind.com/> prendre la version gratuite)

Plus c'est détaillé et organisé par chapitre mieux c'est ;)

C'est la vitrine de votre projet (la première impression ne laisse pas une seconde chance)

- **Le .gitignore**

Ce fichier à la racine évite d'ajouter n'importe quel fichier par mégarde et de polluer votre dépôt de fichiers inutile.

Exemple on ne commit que les fichiers sources du projet, on ne commit en aucun cas des binaires (exécutable) et des fichiers de RUN générés à l'exécution (log/rapport).

Si votre projet est un projet d'automatisation des tests, on exclut d'office tout ce qui est reporting



- PRESENTATION GIT
- **EXERCICES GIT**

EXERCICES : AJOUT DU README

- Ajout du README directement via l'interface Web GitLab

private-cds-sogeti > formation



formation 
Project ID: 28810714



★ Star

0

Invite your team

Add members to this project and start collaborating with your team.

Invite members

The repository for this project is empty

You can get started by cloning the repository or start adding files to it with one of the following options.

Clone ▾

Upload File

New file

Add README

Add LICENSE

Add CHANGELOG

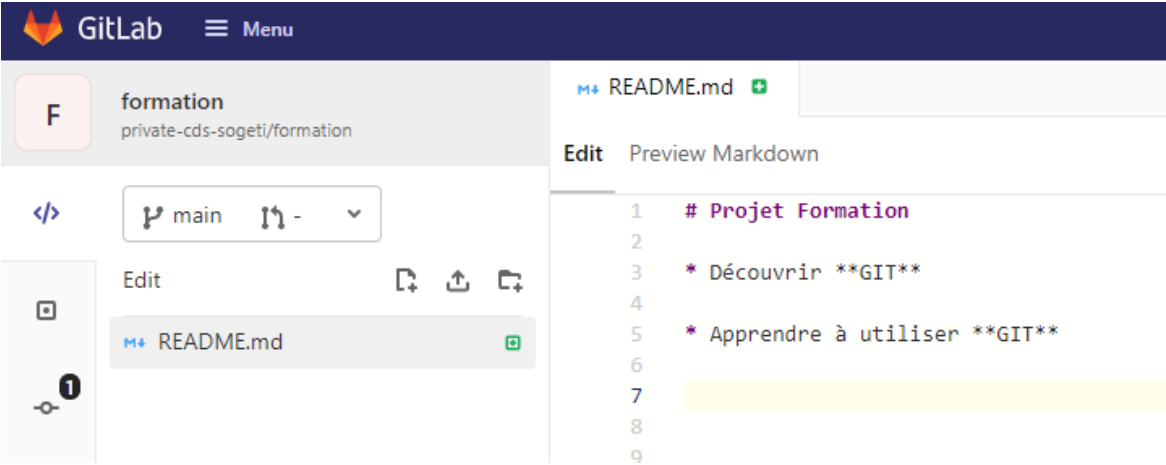
Add CONTRIBUTING

Set up CI/CD

Configure Integrations

EXERCICES : AJOUT DU README

- Saisir ce texte en respectant les caractères markdown, puis faire le commit
- <https://www.markdownguide.org/basic-syntax/>



Name	Last commit	Last update
M+ README.md	Update README.md	16 seconds ago

README.md

Projet Formation

- Découvrir **GIT**
- Apprendre à utiliser **GIT**

- Le rendu sur la page du projet

EXERCICES : CLONER LOCALEMENT LE DEPOT DISTANT

- Depuis Git Bath ou directement en clic droit sur votre explorer
- Se positionner dans votre workspace
- Exemple chemin windows

C:\Users\xxxxxxx\AppData\Local\TESTFACTORY\workspace

- Exemple chemin linux de Git Bash

/c/Users/xxxxxxx/AppData/Local/TESTFACTORY/workspace/

- Saisir cette commande pour vous déplacer dans le workspace

cd /c/Users/xxxxxxx/AppData/Local/TESTFACTORY/workspace/

- Connaitre l'url de votre repo

<https://gitlab.com/private-cds-sogeti/formation.git>

- Cloner votre repo

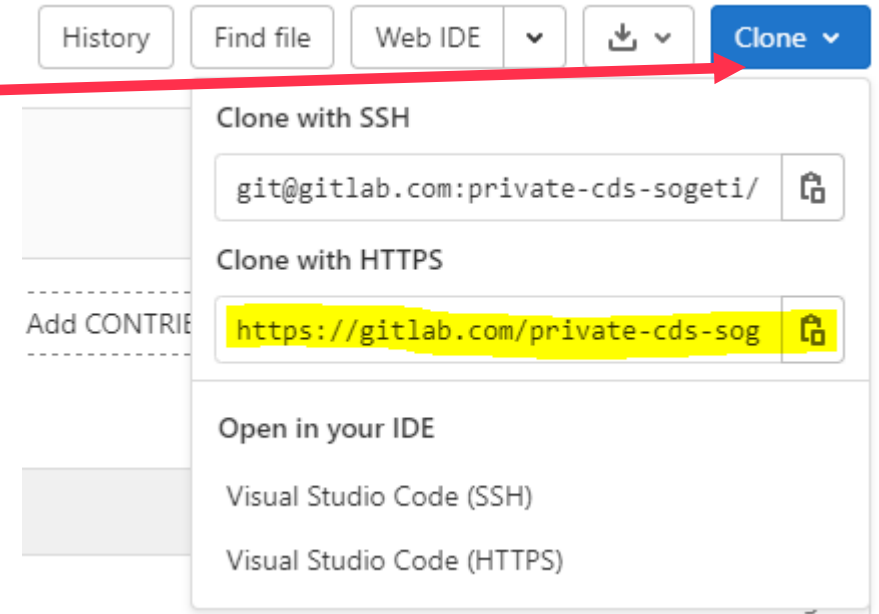
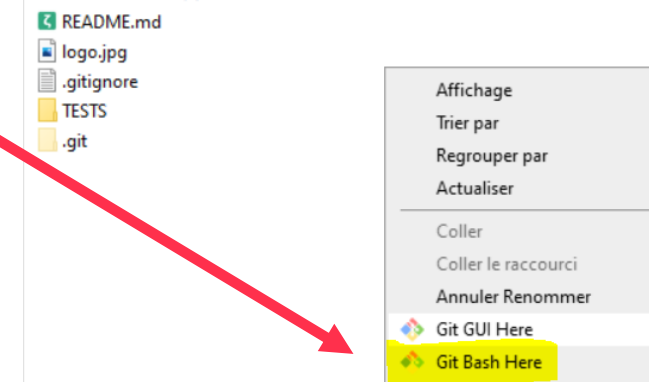
git clone <https://gitlab.com/private-cds-sogeti/formation.git>

- Se placer dans votre repo local

cd formation

- Lister le contenu du répertoire

ll ou ls -l



EXERCICES : AJOUT DU .gitignore

- Depuis votre éditeur de texte

Créer à la racine un fichier .gitignore avec le contenu ci-contre →

```
# tout ignorer
/*
# ne pas ignorer
!.gitignore
!README.md
!logo*
!/TESTS
```

- Depuis Git Bath

```
git status
```

```
git add .gitignore
```

```
git status
```

```
git commit -m "create .gitignore"
```

```
git status
```

- Consulter avant / après le PUSH le contenu du repo via GitLab.com

```
git push
```

EXERCICES : TEST DU .gitignore

- Créer un fichier test.txt à la racine

`git status` → Que se passe-t-il ?

- Déplacer ce fichier dans un nouveau répertoire TESTS

`git status` → Que se passe-t-il ?

- Allez chercher un logo sur internet et enregistrer le à la racine sous le nom logo.png (ou jpeg)

`git status`

`git add *`

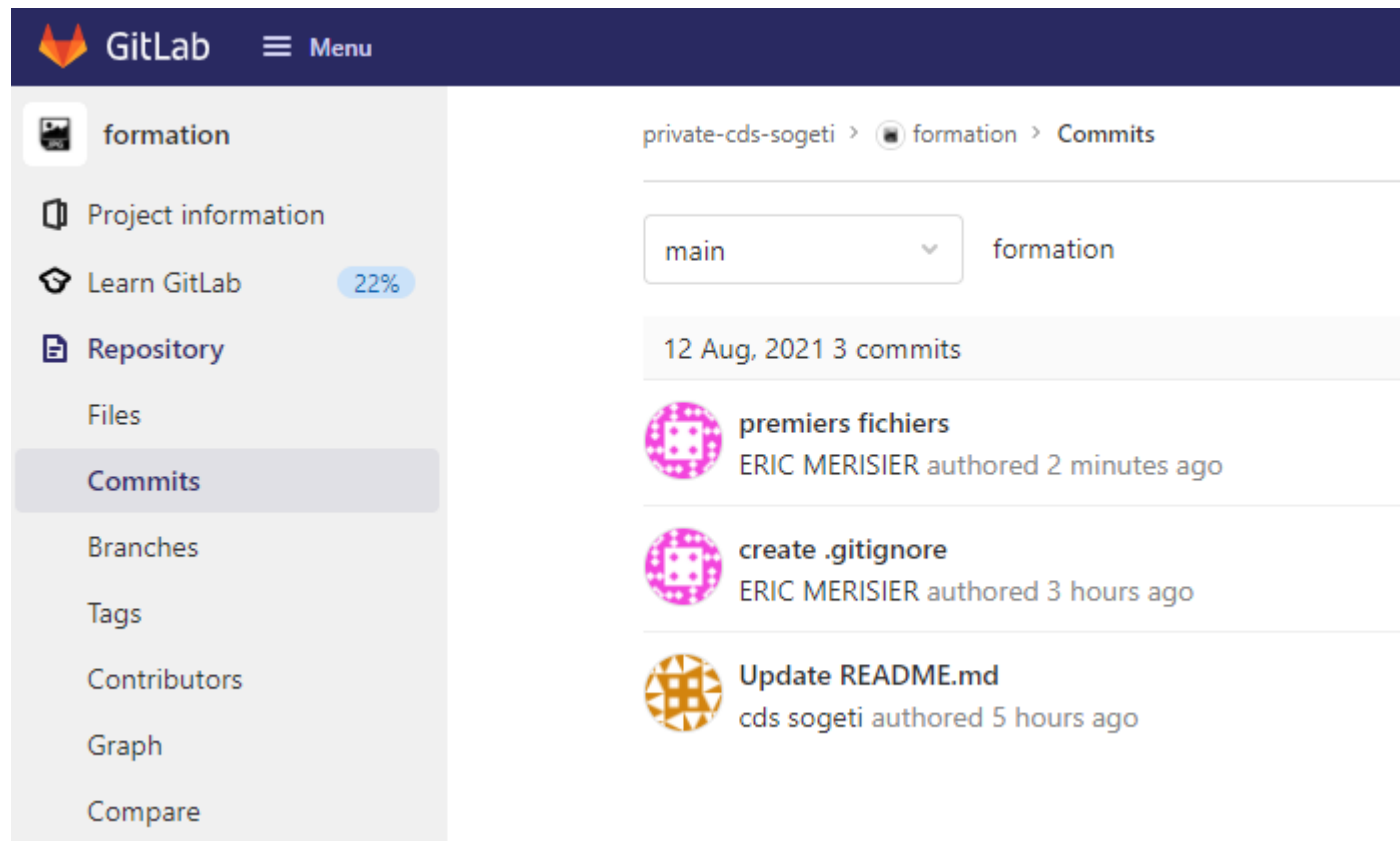
`git commit -m "ajout premiers fichiers"`

`git push`

→ Que constate-t-on sur gitlab.com ?

EXERCICES : Visualiser

- L'historique des commits



The screenshot displays the GitLab web interface for a repository named 'formation'. The left sidebar contains a navigation menu with options: 'formation' (selected), 'Project information', 'Learn GitLab' (with a '22%' badge), 'Repository', 'Files', 'Commits' (highlighted), 'Branches', 'Tags', 'Contributors', 'Graph', and 'Compare'. The main content area shows the breadcrumb 'private-cds-sogeti > formation > Commits'. Below this, there is a dropdown menu set to 'main' and the repository name 'formation'. A summary bar indicates '12 Aug, 2021 3 commits'. The commit history lists three commits: 1. 'premiers fichiers' by ERIC MERISIER, authored 2 minutes ago, with a pink commit icon. 2. 'create .gitignore' by ERIC MERISIER, authored 3 hours ago, with a pink commit icon. 3. 'Update README.md' by cds sogeti, authored 5 hours ago, with an orange commit icon.

EXERCICES : Visualiser

- Le détail du commit

 **GitLab** Menu

 formation

 Project information

 Learn GitLab 22%

 Repository

Files

Commits

Branches

Tags

Contributors

Graph

Compare

 Issues 0

 Merge requests 0

 CI/CD

premiers fichiers

 parent 407b48ec Pmain

 No related merge requests found

Changes 2

Showing 2 changed files with 0 additions and 0 deletions

 TESTS/test.txt  0 → 100644

This diff

 logo.jpg  0 → 100644

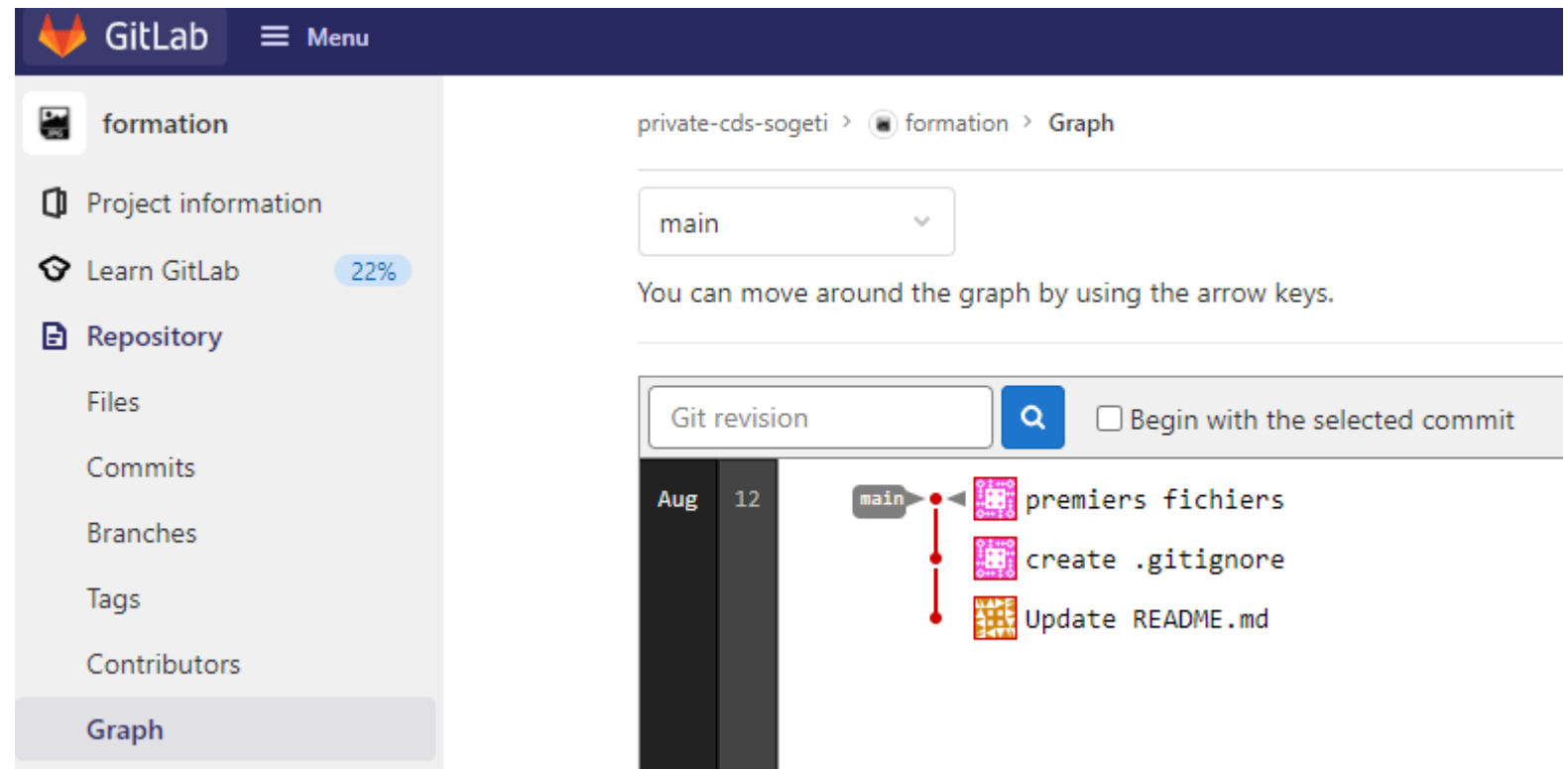
Formation Robot Framework | 2023



© 2023 Sogeti. All rights reserved. 21

EXERCICES : Visualiser

- La vue graph
- Qu'un seul arbre car il n'y a que la branche main (master)



The screenshot displays the GitLab web interface for a repository named 'formation' under the path 'private-cds-sogeti'. The left sidebar shows the navigation menu with options: Project information, Learn GitLab (22%), Repository (selected), Files, Commits, Branches, Tags, Contributors, and Graph. The main content area shows the 'Graph' view for the 'main' branch. A dropdown menu at the top of the graph area is set to 'main'. Below this, a text instruction states: 'You can move around the graph by using the arrow keys.' The graph itself shows a vertical sequence of three commits on the 'main' branch, starting from a base point. The commits are represented by colored squares (pink and orange) and labeled with their descriptions: 'premiers fichiers', 'create .gitignore', and 'Update README.md'. A calendar widget on the left of the graph shows the date 'Aug 12'.

CONCLUSIONS

- C'est simple ?
 - Tant qu'on est seul OUI car lors du PUSH la fusion (merge) de la branche locale et distante est sans ambiguïté.
 - Le travail collaboratif risque de générer des conflits lorsque vous avez modifié le même fichier dans votre branche locale, lors du push, le merge va détecter un conflit qu'il faudra résoudre
 - Qui a raison ? Les mises à jour de la personne A ou de la personne B ou les deux ont raisons ?
- Git ne prendra aucune décision de merge automatique
- Un outil comme KDIFF proposera les 3 versions
 - Le contenu de la branche de l'utilisateur A
 - Le contenu de la branche de l'utilisateur B
 - Le contenu proposé à la fusion, c'est ici qu'on résout les conflits
 - A vous de faire le ménage sur les bonnes lignes de codes à conserver dans le fichier
 - → **Pour l'exercice de conflit**, il est possible de générer un conflit avec un seul utilisateur s'il travaille sur le même fichier sur 2 branches

LES BRANCHES

- En général la branche main (master) est la branche de production
- Tout nouveaux développements se font sur des branches qui peuvent être temporaires
- On utilise généralement la fonctionnalité de merge request pour fusionner une branche de dev vers la branche main
- Cette démarche a plusieurs avantages
- Je suis isolé pendant le développement de ma fonctionnalité, je peux pousser un travail non terminé pour avoir une sauvegarde coté serveur, sans impact sur la branche de production.
- Je ferai autant de commit que je souhaite sans polluer la branche de production
- Je fais une demande de merge request quand mon travail est terminé et testé
- La personne qui valide le merge request est souvent de la responsabilité d'un tech lead, il fera certainement une revue de code.
- La branche de production aura qu'un seul commit qui est le merge portant le nom de la fonctionnalité développée

EXERCICES : LES BRANCHES

- Création d'une branche dev

`git branch dev`

- Visualiser les branches

`git branch` → en vert la branche active main

- Se positionner sur la branche dev

`git checkout dev`

`git branch` → en vert la branche active dev

- # je peux créer la branche et me positionner dessus en une seule commande
- # `git checkout -b dev`
- Se positionner sur la branche main

`git checkout main`

- Modifier le fichier TESTS/test.txt

`echo code origine >> TESTS/test.txt`

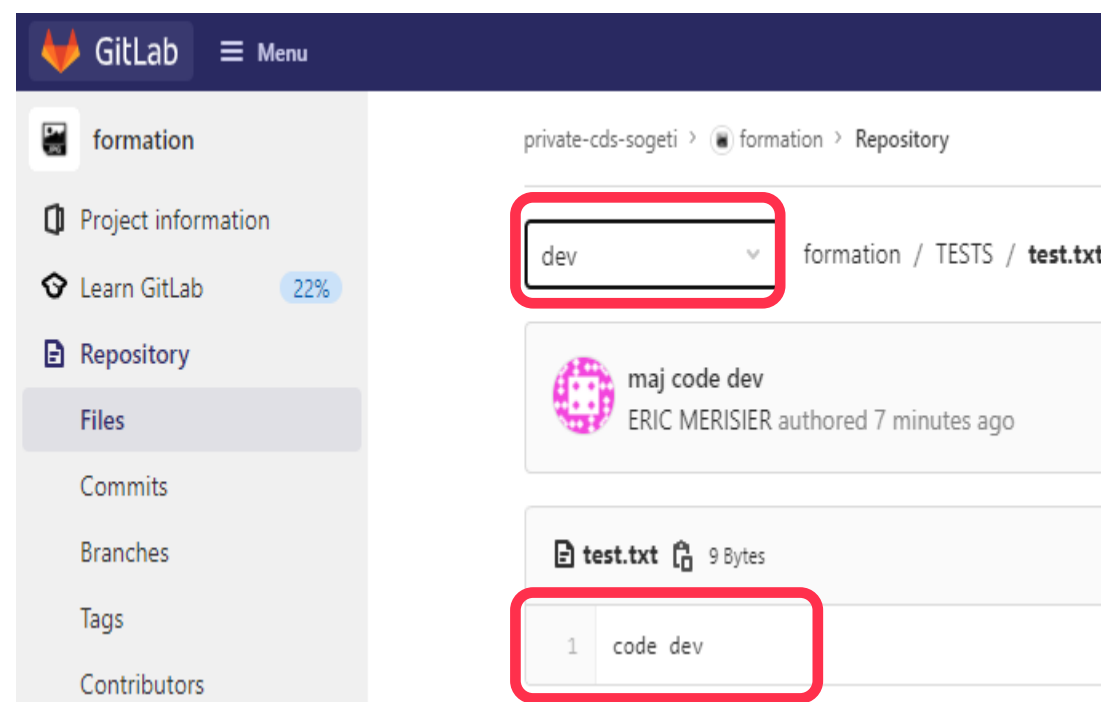
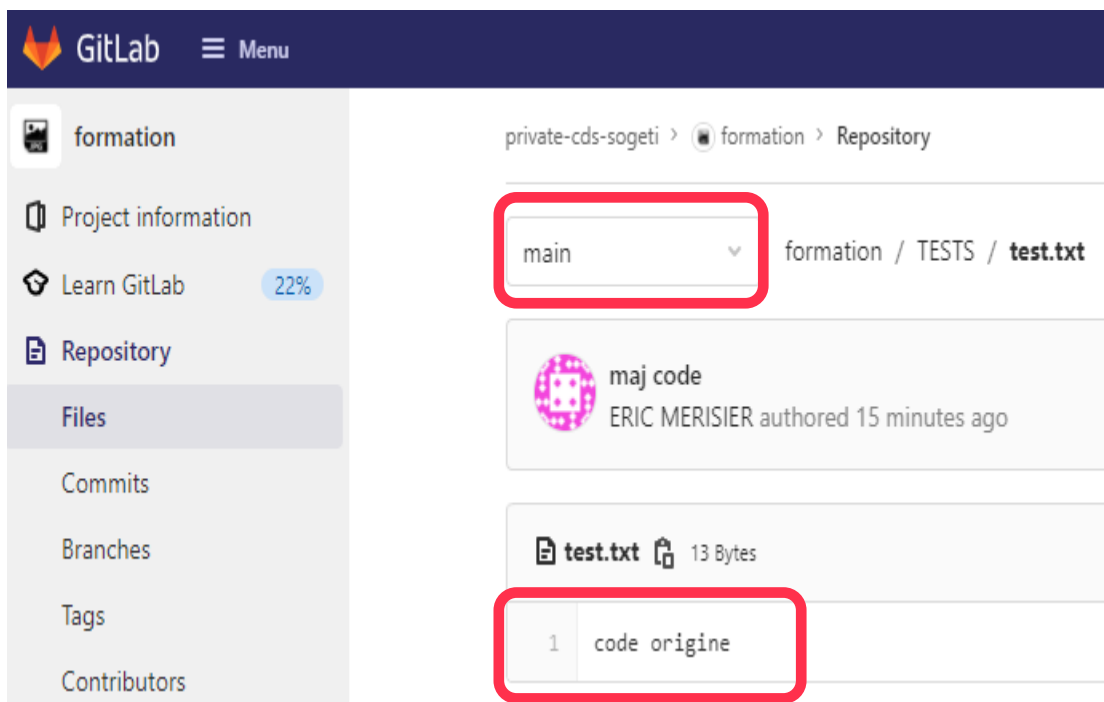
- Faire les étapes pour pousser vos modifications sur le serveur : `add / commit / push`
- Se positionner sur la branche dev

`git checkout dev`

- Consulter le contenu de TESTS/test.txt → que constate-t-on ?

EXERCICES : LES BRANCHES

- Modifier sur la branche dev le fichier TESTS/test.txt
`echo code dev >> TESTS/test.txt`
- Faire les étapes pour pousser vos modifications sur le serveur : add / commit / push
- Lors du premier push sur cette nouvelle branche faire : `git push --set-upstream origin dev`
- Visionner la différence sur GitLab.com



EXERCICES : LES BRANCHES

- On peut voir ici par le nom du commit que la MAJ du fichier test.txt a été fait sur la branche main après la création de la branche dev qui a aussi modifié ce fichier
- La branche dev ne contient donc pas le dernier commit de main sur ce fichier
- Il y aura donc un conflit de merge sur ce fichier

GitLab Menu

formation

private-cds-sogeti > formation > Graph

main

You can move around the graph by using the arrow keys.

Git revision ☐ Begin with the selected commit

Aug 13

dev maj code dev

main maj code

12

premiers fichiers

create .gitignore

Update README.md

EXERCICES : MERGER DEUX BRANCHES

- Se positionner sur main

`git checkout main`

- Merger dev sur main

`git merge dev`

- On annule la tentative de merge

`git merge --abort`

```
$ git merge dev
Auto-merging TESTS/test.txt
CONFLICT (content): Merge conflict in TESTS/test.txt
Automatic merge failed; fix conflicts and then commit the result.
```

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

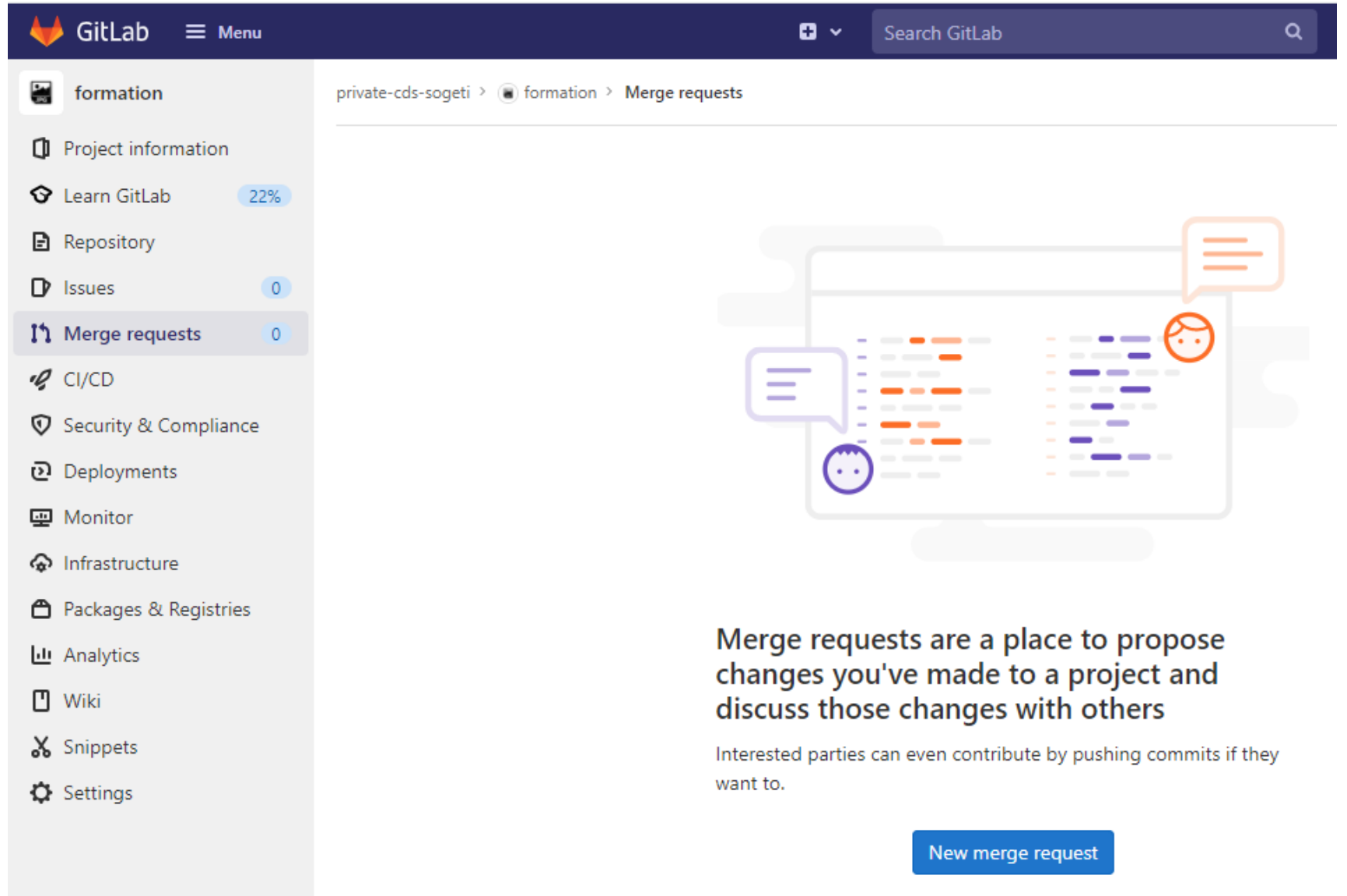
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   TESTS/test.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

EXERCICES : FAIRE UN MERGE REQUEST

- Depuis GitLab.com



The screenshot shows the GitLab web interface. The top navigation bar includes the GitLab logo, a menu icon, a search bar, and a plus icon. The left sidebar contains a list of navigation items: 'formation' (selected), 'Project information', 'Learn GitLab' (22%), 'Repository', 'Issues' (0), 'Merge requests' (0), 'CI/CD', 'Security & Compliance', 'Deployments', 'Monitor', 'Infrastructure', 'Packages & Registries', 'Analytics', 'Wiki', 'Snippets', and 'Settings'. The main content area shows the breadcrumb 'private-cds-sogeti > formation > Merge requests'. Below the breadcrumb is a large illustration of a computer monitor displaying a merge request interface with two columns of code (one orange, one purple) and speech bubbles. Below the illustration, the text reads: 'Merge requests are a place to propose changes you've made to a project and discuss those changes with others'. Below this text is a smaller line: 'Interested parties can even contribute by pushing commits if they want to.' At the bottom right of the main content area is a blue button labeled 'New merge request'.

EXERCICES : FAIRE UN MERGE REQUEST

- Depuis GitLab.com

The screenshot shows the GitLab web interface for creating a new merge request. The top navigation bar is dark blue with the GitLab logo, a menu icon, and a search bar. The left sidebar contains a list of project navigation items: 'formation' (selected), 'Project information', 'Learn GitLab' (22%), 'Repository', 'Issues' (0), 'Merge requests' (0), 'CI/CD', 'Security & Compliance', 'Deployments', and 'Monitor'. The main content area is titled 'New merge request' and shows the breadcrumb 'private-cds-sogeti > formation > Merge requests > New'. It features two columns: 'Source branch' and 'Target branch'. The 'Source branch' column shows a dropdown for 'private-cds-sogeti/formation' and a selected branch 'dev' (highlighted in yellow). Below this, a commit preview shows a commit by 'maj code' (ERIC MERISIER) titled 'maj code dev' from 42 minutes ago, with commit ID '54090740'. The 'Target branch' column shows a dropdown for 'private-cds-sogeti/formation' and a selected branch 'main' (highlighted in yellow). Below this, a commit preview shows a commit by 'maj code' (ERIC MERISIER) titled 'maj code' from 49 minutes ago. A blue button labeled 'Compare branches and continue' is located at the bottom of the source branch section.

EXERCICES : FAIRE UN MERGE REQUEST

- Depuis GitLab.com

New merge request

From dev into main [Change branches](#)

Title

maj code dev

Start the title with `draft:` to prevent a merge request that is a work in progress from being merged before it's ready.
Add [description templates](#) to help your contributors communicate effectively!

Description

Write Preview

B *I* ” </> ↗

Describe the goal of the changes and what reviewers should be aware of.

Markdown and [quick actions](#) are supported

Assignee

cds sogeti

Reviewer

Unassigned

Approvals are optional.
[Approval rules](#)

Milestone

Milestone

Labels

Labels

Merge options

☒ Delete source branch when merge request is accepted.
☐ Squash commits when merge request is accepted. ?

Create merge request

Cancel



EXERCICES : FAIRE UN MERGE REQUEST

- Depuis GitLab.com

maj code dev

Overview 0

Commits 1

Changes 1

Request to merge dev into main

The source branch is 1 commit behind the target branch

Open in Web IDE

Check out branch

?

No pipeline

Add the .gitlab-ci.yml file to create one.

Are you adding technical debt or code vulnerabilities?

Use CI pipelines to test your code by simply adding a GitLab CI configuration file to your project. It only takes a minute to make your code more secure and robust.

Show me how to add a pipeline

Approve

Approval is optional

Merge

There are merge conflicts

Resolve conflicts

Merge locally

0

0

Oldest first

Show all activity

cds sogeti

@cds.sogeti assigned to @cds.sogeti just now

Write

Preview

B

I

”

</>

Write a comment or drag your files here...

Formation Robot Framework | 2023



© 2023 Sogeti. All rights reserved. 32

EXERCICES : FAIRE UN MERGE REQUEST

- Depuis GitLab.com
- Le mode « interactive » me permet de choisir que l'un ou l'autre modification
- Si on souhaite modifier librement le fichier, on passe en « Edit inline »

private-cds-sogeti > formation > Merge requests

Open

Created 1 minute ago by  cds sogeti

Owner

Edit

Mark as draft

maj code dev

Showing 1 conflict between dev and main

Inline

Side-by-side

TESTS/test.txt

Interactive mode

Edit inline

View file @ 5409074

HEAD//our changes

code dev

code origine

origin//their changes

Use ours

Use theirs

Resolve conflicts on source branch

You can resolve the merge conflict using either the Interactive mode, by choosing **Use ours** or **Use theirs** buttons, or by editing the files directly. Commit these changes into **dev**

Commit message

Merge branch 'main' into 'dev'

Conflicts:
TESTS/test.txt

Commit to source branch

Cancel

EXERCICES : FAIRE UN MERGE REQUEST

- Depuis GitLab.com
- Les lignes avec les chevrons seront à supprimer, ils indiquent la différence

TESTS/test.txt

Interactive mode

Edit inline

1

<<<<<<< TESTS/test.txt

Ligne qui provient de la gauche donc de la branche dev

2

code dev

3

=====

4

code origine

5

>>>>>>> TESTS/test.txt

Ligne qui provient de la droite donc de la branche main

6

- Le résultat si je veux conserver les deux lignes

TESTS/test.txt

1

code dev

2

code origine

3

Commit message

Merge branch 'main' into 'dev'

Conflicts:

TESTS/test.txt

Commit to source branch

Cancel

Comme message de commit de merge, il est préférable de mettre le nom de l'évolution/correction qui vous a amené à créer cette branche de développement

- Puis je commit

EXERCICES : FAIRE UN MERGE REQUEST

- Depuis GitLab.com
- Mon bouton Merge est désormais actif
- La branche DEV sera supprimée du serveur après la fusion

maj code dev

Overview 0 Commits 1 Changes 1

 Request to merge dev into main

 No pipeline [Add the .gitlab-ci.yml file](#) to create one.

Are you adding technical debt or code vulnerabilities?

Use [CI pipelines to test your code](#) by simply adding a GitLab CI configuration file to your project. It only takes a minute to make your code more secure and robust.

Show me how to add a pipeline

 Approval is optional 

 ☒ Delete source branch ☐ Squash commits 

EXERCICES : FAIRE UN MERGE REQUEST

- Depuis GitLab.com

The screenshot displays the GitLab web interface for a project named 'formation' under the namespace 'private-cds-sogeti'. The left sidebar contains navigation links: 'formation', 'Project information', 'Learn GitLab' (33%), 'Repository', 'Files', 'Commits', 'Branches', 'Tags', 'Contributors', 'Graph' (selected), 'Compare', 'Issues' (0), and 'Merge requests' (0). The main content area shows the 'Graph' view for the 'main' branch. A dropdown menu at the top of the graph area is set to 'main'. Below it, a text prompt states: 'You can move around the graph by using the arrow keys.' A search bar labeled 'Git revision' is present, along with a checkbox 'Begin with the selected commit'. The graph itself shows a vertical timeline of commits. The 'main' branch is highlighted in green, starting from a commit on August 12th and ending at a commit on August 13th. The 'dev' branch is shown in red, branching off from 'main' and merging back into 'main'. The commit history for the 'main' branch includes: 'Update README.md', 'create .gitignore', 'premiers fichiers', 'maj code', 'maj code dev', 'Merge branch 'main' into 'dev'', and 'Merge branch 'dev' into 'main''. Each commit is represented by a colored square icon and a text description.

EXERCICES : FAIRE UN MERGE REQUEST

- Depuis git bash
- Notre branche local dev est toujours présente mais n'est plus à jours car elle n'a pas été mergé localement (la branche locale est non mergée car l'opération a été faite sur gitlab)
- Supprimer une branche nom mergée

```
$ git branch -D dev  
Deleted branch dev (was 5409074).
```

```
$ git branch  
dev  
* main
```

EXERCICES : ETRE A JOUR EN LOCAL

- Depuis git bash
- Etrange, le status nous dit que nous sommes à jour, pourtant si je vérifie mon fichier test.txt, il ne comprend pas ma ligne de code provenant de ma branche dev

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

```
$ git diff main origin/main
```

- Faire un **git fetch** pour rafraichir mon repo local avec le distant

```
$ git fetch
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 5 (delta 2), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (5/5), 529 bytes | 13.00 KiB/s, done.
From https://gitlab.com/private-cds-sogeti/formation
 9c8c078..6c0a007  main      -> origin/main
```

- Refaire un **git status**

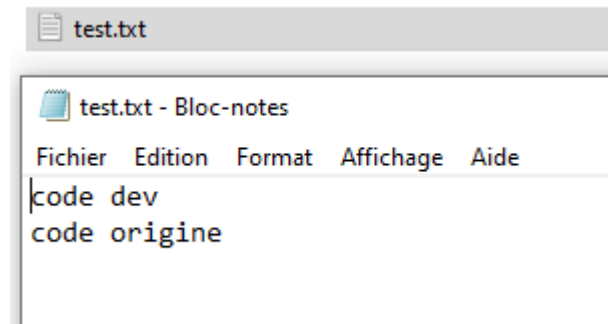
```
$ git diff main origin/main
diff --git a/TESTS/test.txt b/TESTS/test.txt
index 428200b..0477cad 100644
--- a/TESTS/test.txt
+++ b/TESTS/test.txt
@@ -1,2 @@
+code dev
code origine
```

EXERCICES : ETRE A JOUR EN LOCAL

- Depuis git bash
- mon fichier test.txt ne comprend toujours pas ma ligne de code provenant de ma branche dev
- A oui il faut un **git pull** pour mettre à jour le workspace
- (j'aurai pu le faire directement sans faire le **git fetch** intermédiaire)

```
$ git pull
Updating 9c8c078..6c0a007
Fast-forward
 TESTS/test.txt | 1 +
 1 file changed, 1 insertion(+)
```

- Mon fichier est à jour



EXERCICES : ETRE A JOUR EN LOCAL

- Depuis git bash
- Un git log pour confirmer tout ca
- Aucun doute, le numéro de commit est identique entre le local et le distant

```
$ git log
commit 6c0a007c388b1b984f97c525ed9b2bf7597f69a6 (HEAD -> main, origin/main, origin/HEAD)
Merge: 9c8c078 1fb98dc
Author: cds sogeti <cds.sogeti.33@gmail.com>
Date:   Fri Aug 13 07:15:08 2021 +0000

    Merge branch 'dev' into 'main'

    maj code dev

    See merge request private-cds-sogeti/formation!1
```

The screenshot shows the GitLab web interface for the 'formation' repository. The left sidebar contains navigation links: formation, Project information, Learn GitLab (33%), Repository, Files, Commits (selected), Branches, Tags, Contributors, and Graph. The main content area displays the commit history for the 'main' branch. The top commit is 'Merge branch 'dev' into 'main'' by 'cds sogeti' with hash '6c0a007c'. Below it are 'Merge branch 'main' into 'dev'' and 'maj code dev'. A red arrow originates from the commit hash '6c0a007c' in the terminal output above and points to the same hash in the GitLab commit list.

Commit Hash	Author	Message
6c0a007c	cds sogeti	Merge branch 'dev' into 'main'
1fb98dc0	cds sogeti	Merge branch 'main' into 'dev'
54090740	ERIC MERISIER	maj code dev

About Sogeti

Part of the Capgemini Group, Sogeti makes business value through technology for organizations that need to implement innovation at speed and want a local partner with global scale. With a hands-on culture and close proximity to its clients, Sogeti implements solutions that will help organizations work faster, better, and smarter. By combining its agility and speed of implementation through a DevOps approach, Sogeti delivers innovative solutions in quality engineering, cloud and application development, all driven by AI, data and automation.

Capgemini is a global leader in partnering with companies to transform and manage their business by harnessing the power of technology. The Group is guided everyday by its purpose of unleashing human energy through technology for an inclusive and sustainable future. It is a responsible and diverse organization of 360,000 team members in more than 50 countries. With its strong 55-year heritage and deep industry expertise, Capgemini is trusted by its clients to address the entire breadth of their business needs, from strategy and design to operations, fueled by the fast evolving and innovative world of cloud, data, AI, connectivity, software, digital engineering, and platforms. The Group reported in 2022 global revenues of €22 billion. Get the Future You Want

Visit us at
www.sogeti.com



This message contains information that may be privileged or confidential and is the property of the Capgemini Group.

Copyright© 2023 Sogeti. All rights reserved.